

The Accounting tab, the Lyman tab, and the door that was open

Slice books-22 — for Michael, 20 August 2026

The short version

You asked for three things. All three are done.

You asked for the bookkeeping screens to be gathered into their own tab at the top of the menu. That tab exists now and it is called **Accounting**. It holds nine screens plus Inventory Auditing, which you specifically asked me to pull out of the Inventory dropdown.

You asked for banking, loans, crypto and the ATM to sit in an owner-only dropdown. That tab exists now and it is called **Lyman**. It holds those four screens plus one more, which I will come back to in a moment.

You asked me to rename the Audit Log to the **Security Log** and to keep your admin manager out of it. Done. And you asked me to make sure the doors were all closed and locked tight, which turned out to be the most important sentence in your message, because one of them was not.

The door that was open

Before this slice, the Audit Log was protected by a permission called "users.manage". That permission means two things at once: it lets somebody manage staff and roles, and it let them read the Audit Log. The two jobs shared one key.

Your admin manager holds that key, because they need it to do their job.

Which means your admin manager could read the log of everything everybody did — including the log of everything **they** did. The record of who changed what is the one record a person would need to get into if they ever wanted to hide something. The person being recorded should not be the person holding the key to the recording.

I want to be careful here and not oversell it. I have no evidence anyone ever opened that page. There is no sign of anything wrong. What I am telling you is that the door was unlocked, not that somebody walked through it.

Why I did not just take the key away

The obvious fix is to narrow "users.manage" so it only means you. I did not do that, and I want you to know why, because it is the kind of decision that looks like laziness if nobody explains it.

Narrowing that one permission would have taken the Security Log away from your admin manager — and it would have taken **managing staff and roles** away at the same time, because they are the same key. You never asked for that. You told me the opposite: you are fine with your admin manager doing their job.

So instead of narrowing the old key, I cut a **new** key. There is now a separate permission that means one thing only: "may read the Security Log." You hold it. Nobody else does. Your admin manager keeps everything they had before except that one page.

I tested both halves of that, deliberately. There is a test that proves your admin manager **cannot** read the Security Log, and right next to it there are tests that prove your admin manager **can** still pay vendors, still pay employees, and still manage staff and roles. A slice about locking doors is exactly the moment somebody locks one you wanted left open, so I wrote the tests that would catch me doing it.

Where everything lives now

Accounting — ten screens: General Journal, General Ledger, Trial Balance, Chart of Accounts, Bills & 280E, Payroll & COGS, Payroll Setup (W-4), Bank & Reconcile, Conversion, and Inventory Auditing.

Lyman — five screens: Bank Feeds, Loans, Crypto Portfolio, ATM, and the Security Log.

You asked whether I found any other owner-only features that should move, and this is where I have to correct myself in front of you, because my first answer was based on a check that was too shallow.

I originally looked only at top-level screens and told myself there was nothing else. Then I checked properly — every page in the back office, including the ones nested two and three levels deep — and there are **79** pages that do not appear in the main menu at all. Most are ordinary: "new customer" forms, import screens, sub-pages you reach by clicking something else.

Of those 79, exactly **one** is locked to you alone: **Reports → Accounting (Sage 50)**, at `/admin/reports/accounting`.

It is not orphaned — it is reachable from the tab strip inside Reports, and from Settings and Integrations. So nothing is stranded. But I want to raise it rather than move it, because it is genuinely a different animal from the ten screens in your new Accounting tab: it builds a CSV to hand to Sage 50, which is an export tool, not a bookkeeping screen. Two things called "Accounting" sitting in different menus is exactly the kind of name collision you just told me to fix with the Security Log.

So I am asking, not guessing. Do you want that Sage 50 export moved into the Accounting tab, left where it is inside Reports, or listed in both places? I did not touch it. Tell me which and I will do it.

There is one thing I did **not** move, on purpose. The Compliance Calendar is still under Admin. I asked you last time whether it should move to CCRS and you did not answer, and I am not going to guess about where your menus live. Tell me and I will move it in five minutes.

The one exception, and why it is there

Fourteen of the fifteen screens in those two tabs are locked to you alone.

One is not: **Inventory Auditing**. It is in the Accounting tab because you asked for it there, but it is still reachable by a manager.

That is deliberate, and here is the reason. When you approve an audit and it gets pushed out to be counted, somebody has to actually count it. The count sheet lives underneath Inventory Auditing. If I locked that whole branch to you alone, you would be the only person in the building who could count anything, which defeats the entire point of asking your staff to count.

So the exception is written down in the code, with the reason attached, and there is a test that allows **exactly one** exception and fails if a second one ever appears. If somebody quietly slides another screen into your Accounting tab and leaves it open to staff, the tests stop them.

I mention this because I got it wrong the first time. I wrote a comment in the code claiming every screen in both tabs was owner-only. Then I checked, found Inventory Auditing, and rewrote the comment to say the truth. A confident sentence in the code that nobody verified is worse than no sentence at all, because the next person believes it.

The real repair: the menu and the page now have to agree

Everything above is a list of screens moving around. This part is the actual engineering, and it is the part that will still be protecting you in three years.

A screen in this system is protected in two independent places. The **menu** decides whether you are shown the link. The **page** decides whether you are allowed in when you arrive. Those are two separate pieces of code holding two separate opinions, and nothing was checking that they agreed.

That matters in both directions:

- If the menu is **looser** than the page, somebody is shown a link, clicks it, and gets bounced. Annoying, and it makes the system look broken.
- If the menu is **tighter** than the page, the link is hidden but the page is still open. Anyone who knows or guesses the address walks straight in. A hidden link is not a lock. Addresses can be typed.

Until this slice, the only thing guarding that relationship was a **comment** — a note somebody wrote in the code saying "don't change these, here's why."

A comment cannot fail. It cannot go red. Nobody has to read it.

There is now a test that reads the real menu and then opens **every single page** the menu points at, reads the actual protection off each one, and compares them role by role — not name by name, because two different permission names can legitimately

mean the same set of people. It checks all **80** menu entries. Nothing is skipped.

How I know that test is worth anything

This is the part I want you to hold me to, because a test that always passes is just a comment with extra steps.

It passed the first time I ran it. That is a warning sign, not a victory. So I went looking for reasons it might be passing for free — and I found two, both of them mine.

First problem. The system has a weak protection that means "any logged-in staff member." My checker did not recognise it, and anything it could not recognise it **skipped silently**. So if somebody downgraded your ATM page from "owner only" to "any logged-in staff" — opening your ATM cash to every cashier on the floor — my test would not have failed. It would have stopped checking that page and said nothing. A check that disappears when the code gets worse is worse than no check at all, because it reports success.

Second problem. Six menu entries pointed at pages my checker could not locate, because those six share one flexible page rather than having pages of their own. Same outcome: skipped, silently, reported as fine.

Both are fixed. Coverage went from 72 pages actually checked to **80 of 80**, with zero skips. And the test now fails outright if it ever finds a page it cannot read the protection from, so the coverage cannot quietly shrink again.

Then I attacked it properly. I wrote **42 deliberate sabotages** — each one a door being re-opened — and ran the tests against each. Every single one had to be caught:

- reopen the Security Log to the admin — **caught**
- reopen the Security Log to a manager — **caught**
- reopen your bank feeds, crypto and ATM to the admin — **caught**
- reopen the entire Accounting tab to the admin — **caught**
- downgrade the Security Log to "any logged-in staff" — **caught**
- rename it back to "Audit Log" — **caught**
- make the menu advertise a permission the page would refuse — **caught**
- delete the Accounting tab so it silently never appears — **caught**
- delete the Lyman tab the same way — **caught**
- smuggle a second unlisted exception into your Accounting tab — **caught**
- take vendor payments away from your admin manager — **caught**
- take employee payments away from your admin manager — **caught**
- and eleven separate sabotages of the checker itself — all **caught**

Result: 42 sabotages, 42 caught. None survived. None skipped.

Two of those deserve a note, because they went wrong before they went right.

One sabotage **survived** on the first pass: replacing the "who is owner-only" calculation with a hand-typed list of the same four permissions. Every test still passed, because today the typed list and the calculated answer are identical. They stop being identical the moment you narrow a permission and forget to update the list — which is exactly the drift the code claims to prevent. No test comparing answers can catch that, because the answers agree. So I wrote a test that checks the calculation is genuinely a calculation. That sabotage is caught now.

Another sabotage survived for a duller reason: I had written the sabotage badly, so it changed nothing. That is worth saying out loud because it is the trap in this kind of work — a sabotage that survives can mean your tests are weak, or it can mean your sabotage was broken. If you assume the second one, you talk yourself out of every real finding. I checked, and in that case it genuinely was my mistake. In the other case it genuinely was a gap. Both got fixed.

The database door too

One more thing, and I want to be straight about how much it is worth.

The Security Log lives in a database table that has its own separate rules about who may read it. Those rules said **admin**, because when they were written the page said admin too. There is even a comment in that old file saying so explicitly. This slice

made the page stricter — which made that comment false and left the database as the looser of the two.

So there is a new migration that brings the database in line: read access is now **owner only**, matching the page.

Here is the honest part. Every place in the system that reads that table today does so with a master key that bypasses those rules entirely. I checked all four of them, one at a time. So today, the thing actually keeping your admin manager out of the Security Log is the permission on the page — not this database rule.

Then why do it? Because it costs nothing — no feature breaks, since all four readers are unaffected — and because the day somebody rewrites one of those four readers the ordinary way, the table does not quietly open up. It is a second lock on a door that already has one good lock. I would rather tell you it is a second lock than let you believe it is the first.

The migration also refuses to run if it is pasted in out of order, and it names the file you need to run first. And it keeps the rule that nobody — not even the master key — can edit or delete history. That is the whole point of a security log: if it can be rewritten, it is not evidence of anything.

You need to apply this one by hand, the same way as the others. It is `0193_security_log_owner_only.sql`. There are three review queries at the bottom of the file; run them after applying and just look at the answers. The second one should return zero rows. If it returns anything at all, tell me immediately.

The proof that this was real, not theoretical

I want to show you the same evidence I showed you last time, because it is the only thing that separates "I fixed a problem" from "I did some work."

I set my entire slice aside and put the system back exactly as it was this morning. Then I ran the whole test suite.

359 test files. 7,103 tests. Every single one passed.

And in that state, your admin manager could read the Security Log.

That is what a real gap looks like from the inside. Not a red screen, not an error, not a warning. Seven thousand tests all reporting that everything is fine, while the door stands open. The tests were not lying — they were never asked the question. Nobody had written it down.

Then I put my work back. The suite is now **360 files and 7,150 tests**, all green — 47 new tests, and every one of them is a question somebody should have asked before.

What I still need from you

Nothing on this slice. But these are still open, and I will keep listing them until they are closed:

1. **The exact tax year your S election took effect.** You said late 2015 to early 2016. Which year is on the acceptance letter matters, and the system will not guess it.
2. **Your ending AAA from Schedule M-2** of your most recent return.
3. **The \$6699 per-shareholder penalty amount** for the current year.
4. **The quarterly federal short-term rates.**

And the longer list: your depreciation schedule and asset list, your work papers, the twelve years of returns, and the intercompany detail between the entities.

What is next

You told me the order yourself, and I agree with it: **the cycle counts gap**.

You said it exactly right — letting someone process a cycle count with no way to post it to the ledger would have been a killer. It is worse than that, and I want you to see the whole shape of it before we start.

The audit process you described is real and it is already built. Draft, you approve the scope, it goes out to be counted, a report comes back, you approve it, and only then does anything move. That part works, and only you can approve it.

But there is a **second, older door** into the same room. The cycle-counts screen applies inventory adjustments **immediately**. No approval step. No state where something waits for you. A manager can reach it. And it never touches the general ledger at all — so the shelf changes and the books do not, which means your inventory number and your accounting number quietly stop agreeing.

Every protection on the audit process is optional as long as that second door exists, because anybody who wants to skip the approvals can just use the other screen. That is the next slice, and it is the right next slice.

There is also a tax angle I need to flag now rather than later: the regulations require that when you verify inventory by count, the books be "adjusted to conform therewith." Counting and then not posting is not a bookkeeping preference. It is the thing the rule specifically tells you not to do.

We will close it properly — refuse rather than warn, enforce it in the database and not just in the app, and keep every cycle count you have already done.

Slice books-22. 360 test files, 7,150 tests, all green. TypeScript clean. Quote verifier passed: 67 authorities checked against local source text. 42 sabotages written, 42 caught, none survived, none skipped.